



Abschlussprüfung Sommer 2025/2026

Fachinformatiker Anwendungsentwicklung
 Projektdokumentation

Kochbuch mit grafischer Benutzeroberfläche und KI-Bildgenerierung

Java-Swing-Anwendung mit Flux-Bildgenerierung

Abgabetermin: Bremen, den 05.07.2026

Prüfungsbewerber:

David Laack
Buntentorsteinweg 428
28201 Bremen



Ausbildungsbetrieb:

cbm GmbH
Wegesende 2-4
28195 Bremen

Inhaltsverzeichnis

1	Projektbeschreibung	2
2	Anforderungsanalyse	2
2.1	Funktionale Anforderungen	2
2.2	Nicht-funktionale Anforderungen	3
3	Technologie-Stack	3
4	Projektziel	3
5	Implementierungsphase	4
5.1	Grundgerüst und Ordnerstruktur	4
5.2	Datenmodell	4
5.3	Speicherung	5
5.4	Grafische Benutzeroberfläche (GUI)	6
5.5	Kernfunktionalitäten	7
5.6	KI-gestützte Bildgenerierung	7
6	Integration von Flux	8
6.1	Was ist Flux?	8
6.2	Anlass der Integration	8
6.3	Technische Umsetzung	8
6.4	Ablauf der Bildgenerierung	8
6.5	Herausforderungen und Grenzen	9
7	Testprotokoll	9
7.1	Testmethodik	9
7.2	Testumgebung	9
7.3	Durchgeführte Tests	9
7.4	Testergebnis	10
8	Fazit	10
8.1	Lessons Learned	10
8.2	Ausblick und Weiterentwicklung	10

1 Projektbeschreibung

Dieses Projekt ist eine Java-Swing-Anwendung mit grafischer Benutzeroberfläche und integrierter API-Anbindung zu einer Flux-Bildgenerierung. Der Benutzer kann Rezepte hinzufügen, bearbeiten, löschen, importieren und suchen.

2 Anforderungsanalyse

2.1 Funktionale Anforderungen

Die folgenden Kernfunktionen wurden als essenziell definiert:

- Anlegen, Bearbeiten und Löschen von Rezepten
- Strukturierte Erfassung von Titel, Kategorie, Zutaten, Zubereitung, Portionen und Zubereitungszeit
- Suche nach Rezepten (nach Titel und Zutaten)
- Filterung nach Kategorien
- Zu jedem Rezept kann ein Bild hinzugefügt werden
- Rezepte werden dauerhaft gespeichert

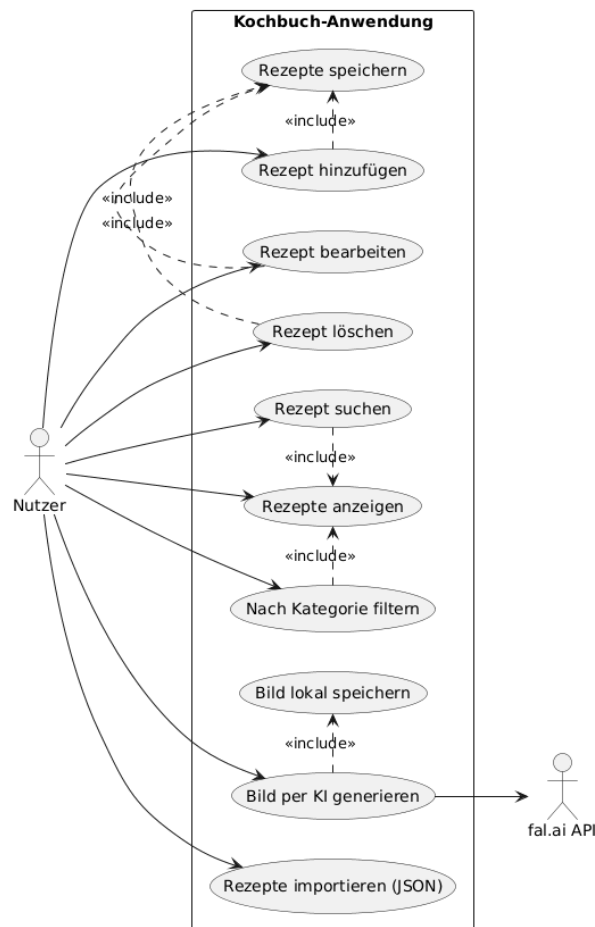


Abbildung 1: UML-Anwendungsfalldiagramm (Use-Case-Diagramm)

2.2 Nicht-funktionale Anforderungen

- Plattformunabhängigkeit durch Java
- Intuitive und übersichtliche Benutzeroberfläche
- Schnelle Reaktionszeit der GUI auch während der Bildgenerierung
- Erweiterbarkeit durch klare Codestruktur (MVC-Prinzip)

3 Technologie-Stack

Das Projekt wurde vollständig in Java entwickelt. Im Folgenden wird der verwendete Technologie-Stack detailliert aufgeführt:

Programmiersprache und Kerntechnologien

- Java 26
- Java Swing – Framework für die grafische Benutzeroberfläche (GUI)
- Gson (Google) – JSON-Parsing für den Import von Rezepten

Bildgenerierung

- Flux Dev/Pro (Black Forest Labs) – KI-Modell zur Text-to-Image-Generierung
- fal.ai API – Schnittstelle zur Nutzung des Flux-Modells

Entwicklungsumgebung

- Eclipse IDE mit integrierten Java Development Kit (JDK) 26
- Git – Versionsverwaltung

Sonstiges

- LaTeX – Erstellung der Projektdokumentation
- TeXstudio – LaTeX-Editor
- Grok (xAI) - zur Behebung von Bugs und allgemeiner Hilfestellung

4 Projektziel

Ziel dieses Projektes ist die Entwicklung einer plattformunabhängigen Desktop-Anwendung zur Verwaltung eines digitalen Kochbuchs. Die Anwendung soll es dem Nutzer ermöglichen, eigene Rezepte strukturiert zu erfassen, zu verwalten und wiederzufinden. Im Kern soll die Software folgende Funktionen bieten:

- Anlegen, Bearbeiten und Löschen von Rezepten mit Angaben zu Titel, Kategorie, Zutaten, Zubereitung, Portionen und Zubereitungszeit
- Persistente Speicherung der Rezepte
- Suche nach Rezepten anhand von Titel oder enthaltenen Zutaten
- Filterung nach Kategorien
- Import von Rezepten aus externen JSON-Dateien
- Übersichtliche grafische Benutzeroberfläche (GUI) basierend auf Java Swing

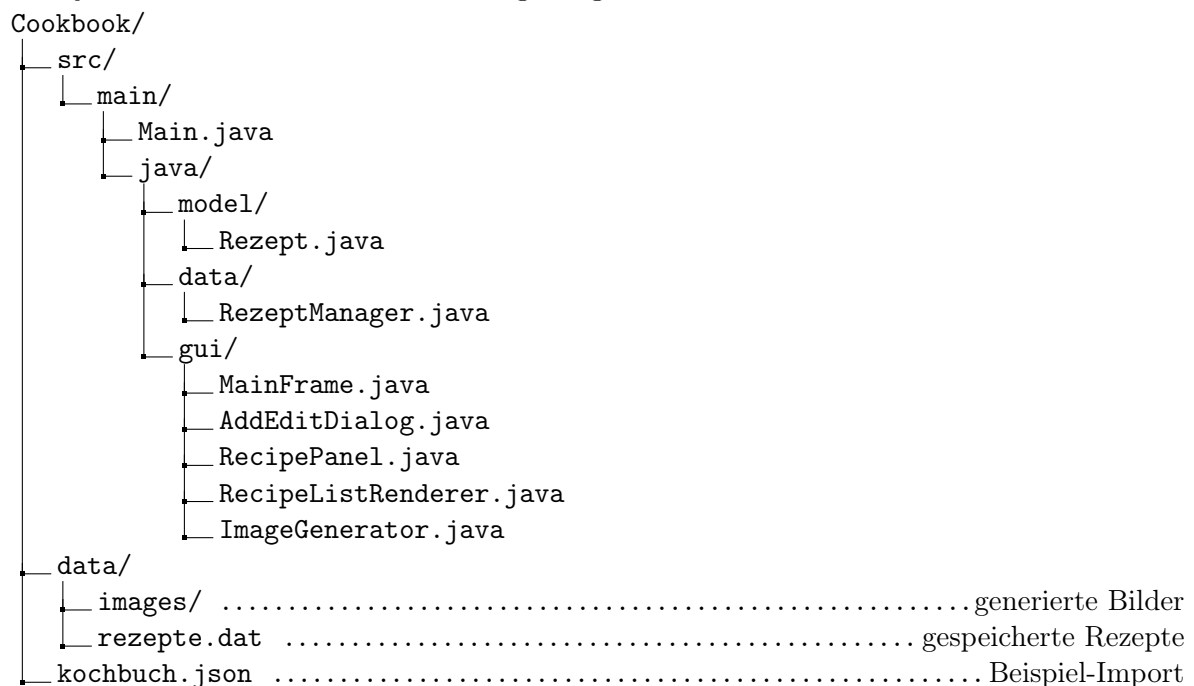
Ursprünglich war geplant, Bilder zu den Rezepten über eine URL-Eingabe zu verlinken, um eine visuelle Darstellung zu ermöglichen. Im Verlauf der Entwicklung wurde diese Funktionalität um eine optionale KI-gestützte Bildgenerierung erweitert, welche auf Basis des Rezepttitels automatisch ein passendes Bild erzeugt und lokal speichert. Das übergeordnete Ziel des Projektes ist es, eine benutzerfreundliche, funktionale und erweiterbare Anwendung zu schaffen, die sowohl für den privaten Gebrauch als auch als Lernprojekt zur Vertiefung von Java-Kenntnissen (insbesondere GUI-Entwicklung, Dateihandling, Serialisierung und API-Anbindung) dient.

5 Implementierungsphase

5.1 Grundgerüst und Ordnerstruktur

Zu Beginn der Umsetzung wurde die grundlegende Paketstruktur (model, data, gui) festgelegt. Diese klare Trennung nach dem MVC-Prinzip (Model-View-Controller) diente der besseren Übersichtlichkeit und Wartbarkeit des Codes. Die Trennung ermöglichte eine saubere Aufteilung der Verantwortlichkeiten und erleichterte die spätere Erweiterung des Projekts.

Die Projektstruktur ist letztendlich wie folgt aufgebaut:



5.2 Datenmodell

Im nächsten Schritt wurde die zentrale Model-Klasse `Rezept.java` implementiert. Diese enthält alle relevanten Attribute eines Rezepts (Titel, Kategorie, Zutaten, Zubereitung, Portionen, Zubereitungszeit, Bildpfad und Notizen). Die Klasse wurde mit geeigneten Getter- und Setter-Methoden sowie einem Konstruktor ausgestattet.

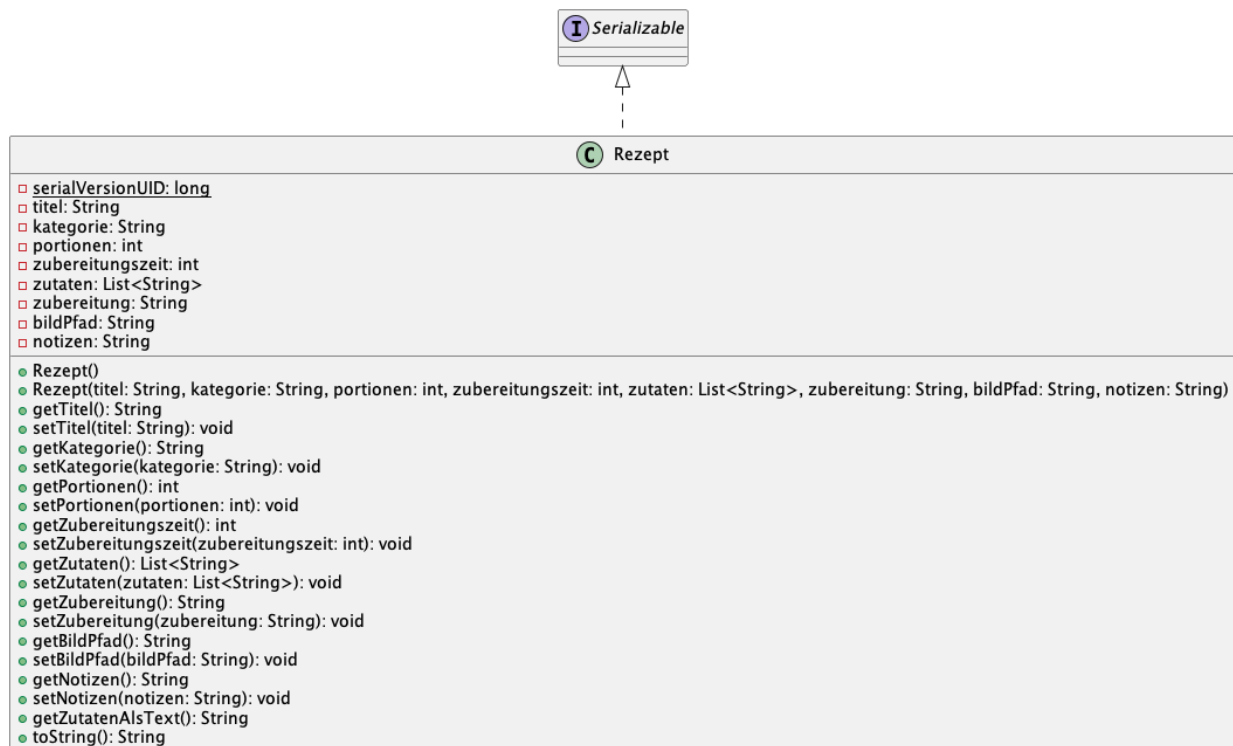


Abbildung 2: UML-Diagramm der Rezept-Klasse

5.3 Speicherung

Zur dauerhaften Speicherung der Rezepte wurde die Klasse **RezeptManager** entwickelt. Diese übernimmt die Verantwortung für das Hinzufügen, Entfernen, Aktualisieren und Laden der Rezepte. Die Daten werden mittels Java-Serialisierung (**ObjectOutputStream** und **ObjectInputStream**) in einer Binärdatei (**rezepte.dat**) gespeichert. Diese Lösung wurde gewählt, da sie einfach zu implementieren ist und eine schnelle Persistenz ohne zusätzliche Datenbank ermöglicht. Gleichzeitig wurde eine Duplikatserkennung nach Rezepttitel integriert.

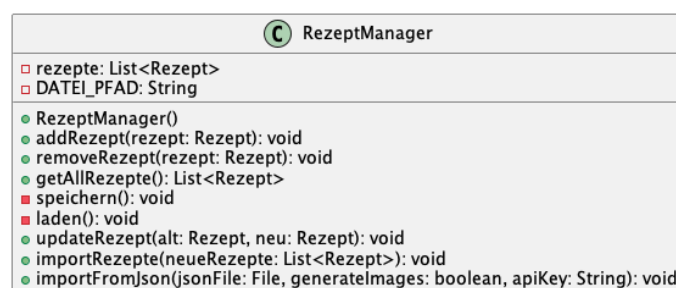


Abbildung 3: UML-Diagramm der RezeptManager-Klasse

5.4 Grafische Benutzeroberfläche (GUI)

Die Benutzeroberfläche wurde mit Java Swing realisiert. Im Zentrum steht die MainFrame-Klasse, welche die Hauptansicht mit Rezeptliste, Suchfunktion und Filtermöglichkeiten bereitstellt.

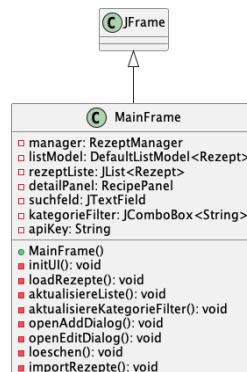


Abbildung 4: UML-Diagramm der MainFrame-Klasse

Der Detailbereich wurde in der Klasse RezeptPanel implementiert.

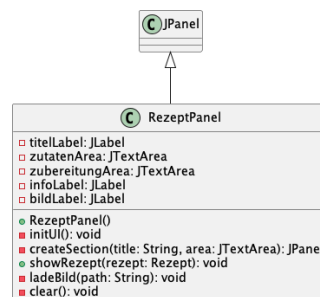


Abbildung 5: UML-Diagramm der RezeptPanel-Klasse

Für das Anlegen und Bearbeiten von Rezepten wurde ein modaler Dialog (AddEditDialog.java) entwickelt.

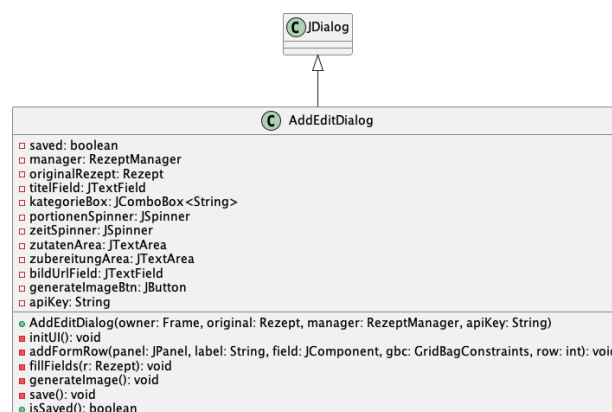


Abbildung 6: UML-Diagramm der AddEditDialog-Klasse

Besonderer Wert wurde auf eine übersichtliche, intuitive Bedienung und ein ansprechendes Layout gelegt.

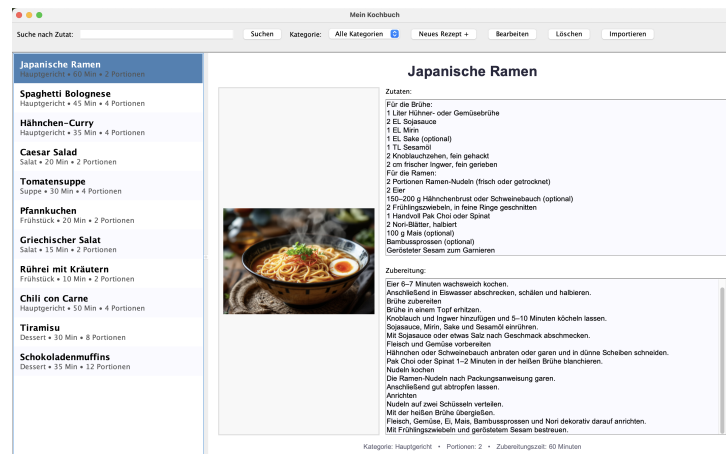


Abbildung 7: Hauptfenster der Anwendung

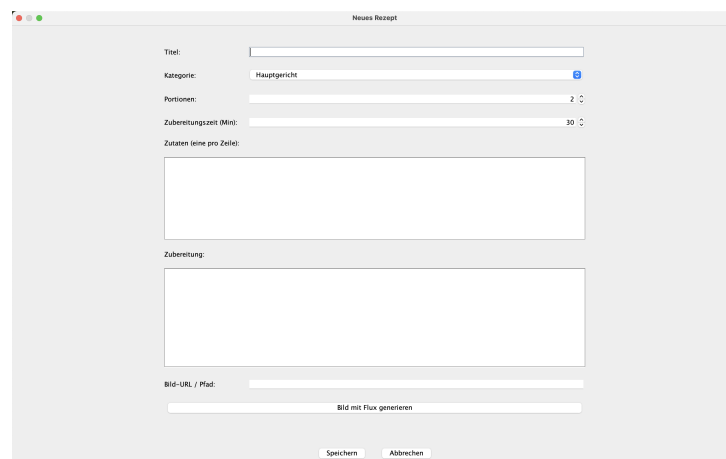


Abbildung 8: Fenster zum Hinzufügen/Ändern eines Rezepts

5.5 Kernfunktionalitäten

Es wurden alle grundlegenden CRUD-Operationen (Create, Read, Update, Delete) implementiert. Darüber hinaus wurden eine erweiterte Suchfunktion (nach Titel und Zutaten) sowie ein Kategoriefilter realisiert. Die Import-Funktionalität ermöglicht das Einlesen von Rezepten aus JSON-Dateien mit automatischer Duplikatserkennung.

5.6 KI-gestützte Bildgenerierung

Als Erweiterung wurde eine Anbindung an die Flux-Bildgenerierungs-API (fal.ai) integriert. Über einen Button im Bearbeitungsdialog kann der Nutzer zu einem Rezepttitel automatisch ein Bild generieren lassen. Die generierten Bilder werden lokal im data/images/-Ordner gespeichert und dem Rezept zugeordnet. Diese Funktion stellt eine wesentliche visuelle Aufwertung der Anwendung dar.

6 Integration von Flux

6.1 Was ist Flux?

Flux ist ein von Black Forest Labs entwickeltes Text-to-Image-Modell. Es zählt zu den modernsten Open-Source-Bildgenerierungsmodellen. Im Rahmen dieses Projekts wurden die Varianten Flux Dev und Flux Pro über die API von fal.ai genutzt.

6.2 Anlass der Integration

Ursprünglich war geplant, Bilder zu den Rezepten ausschließlich über eine URL-Eingabe zu integrieren. Diese Lösung erwies sich jedoch in der Praxis als problematisch, da viele frei verfügbare Bild-URLs nach einiger Zeit nicht mehr erreichbar wären und Urheberrechtefragen aufwarfen. Zudem erforderte diese Methode vom Nutzer, selbst passende Bilder zu suchen und die URLs manuell einzufügen – ein Prozess, der zeitaufwendig und nicht benutzerfreundlich war. Aus diesem Grund wurde die Idee einer automatischen Bildgenerierung mittels Künstlicher Intelligenz entwickelt. Ziel war es, dem Nutzer die Möglichkeit zu geben, mit nur einem Klick ein individuelles, zum Rezept passendes Bild zu erzeugen, ohne auf externe Quellen angewiesen zu sein.

6.3 Technische Umsetzung

Die Anbindung erfolgt über HTTP-Requests an die fal.ai API. Die zentrale Klasse ImageGenerator übernimmt folgende Aufgaben:

- Erstellung eines optimierten Prompts aus dem Rezepttitel
- Senden einer POST-Anfrage an die Flux-API
- Verarbeitung der JSON-Antwort (Extraktion der Bild-URL)
- Herunterladen und lokales Speichern des Bildes im Verzeichnis data/images/
- Rückgabe des Dateipfads an das Rezept

Die Bildgenerierung läuft asynchron in einem separaten Thread, um die Benutzeroberfläche während des Generierungsprozesses (ca. 3-8 Sekunden) nicht zu blockieren.

6.4 Ablauf der Bildgenerierung

1. Der Nutzer klickt auf „Bild mit Flux generieren“
2. Der Rezepttitel wird in einen detaillierten Prompt umgewandelt
3. Die Anfrage wird an die fal.ai API gesendet
4. Das zurückgegebene Bild wird heruntergeladen und lokal gespeichert
5. Der Pfad zum Bild wird im Rezept-Objekt gespeichert und im Detailbereich angezeigt

6.5 Herausforderungen und Grenzen

Die Qualität der generierten Bilder hängt stark vom verwendeten Modell (Flux Dev oder Flux Pro) und der Qualität des Prompts ab. Besonders bei spezifisch regionalen oder traditionellen Gerichten (z.B. Käsespätzle, Maultaschen) liefert Flux ungenaue Ergebnisse, da das Modell primär auf englischsprachigen und internationalen Küchen trainiert wurde.



Abbildung 9: Flux Dev: 'Rührei mit Kräutern'

7 Testprotokoll

7.1 Testmethodik

Das System wurde mittels manueller Tests überprüft. Der Fokus lag auf Funktionalität, Usability und Stabilität.

7.2 Testumgebung

- **Betriebssystem:** macOS Sequoia Version 15.6.1
- **Java-Version:** Java 26
- **Entwicklungsumgebung:** Eclipse IDE

7.3 Durchgeführte Tests

ID	Testfall	Beschreibung	Status
T01	Rezept anlegen	Neues Rezept mit allen Feldern anlegen	✓ Erfolgreich
T02	Rezept bearbeiten	Bestehendes Rezept erfolgreich ändern	✓ Erfolgreich
T03	Rezept löschen	Rezept aus der Liste löschen	✓ Erfolgreich
T04	Suche	Suche nach Titel und Zutat	✓ Erfolgreich
T05	Kategoriefilter	Filter nach Kategorie	✓ Erfolgreich
T06	JSON-Import	Rezepte aus JSON-Datei importieren	✓ Erfolgreich
T07	Bildgenerierung	Bild zu Rezept mit Flux generieren	✓ Erfolgreich
T08	Persistenz	Programm schließen und neu starten	✓ Erfolgreich
T09	Fehlerfall	Versuch, Rezept ohne Titel zu speichern	✓ Erfolgreich

Tabelle 1: Übersicht der durchgeführten Tests

7.4 Testergebnis

Das System ist funktionsfähig und erfüllt die definierten Anforderungen.

Fazit

8.1 Lessons Learned

Ein zentraler Lerneffekt war die Bedeutung einer klaren Architektur. Die konsequente Trennung in Model, Data und GUI hat die Wartbarkeit und Erweiterbarkeit des Projekts erheblich erleichtert. Besonders wertvoll war die Erfahrung mit asynchroner Programmierung bei der Integration der KI-Bildgenerierung, da langlaufende API-Aufrufe die Benutzeroberfläche nicht blockieren dürfen. Weiterhin hat sich gezeigt, dass externe Abhängigkeiten (wie API-Services) sorgfältig geplant werden müssen. Die anfängliche Idee, Bilder lediglich über URLs einzubinden, erwies sich als instabil. Die Umstellung auf eine KI-basierte Generierung hat zwar die Komplexität erhöht, gleichzeitig aber zu einer deutlich höheren Autarkie und Benutzerfreundlichkeit geführt.

8.2 Ausblick und Weiterentwicklung

Das vorliegende Projekt bietet eine solide Grundlage für weitere Erweiterungen. Mögliche zukünftige Entwicklungen sind:

- Implementierung einer Export-Funktion (z.B. als PDF)
- Hinzufügen einer Favoriten-Funktion und persönlicher Bewertungen
- Statistik-Funktionen (z.B. am häufigsten gekochte Kategorien)
- Unterstützung für mehrere Benutzerprofile
- Verbesserung der Bildgenerierung durch fortschrittlichere Prompts oder leistungsfähigere KI-Modelle

Hinweis zur Ausführung der Anwendung: Der Quellcode wird ohne gültigen API-Key eingereicht, da dieser Zugriff auf kostenpflichtige Credits gewährt. Für eine vollständige Ausführung der KI-Bildgenerierung ist ein eigener API-Key von fal.ai erforderlich, welcher in der Klasse `MainFrame.java` an der entsprechend gekennzeichneten Stelle einzutragen ist.